Cloud Computing

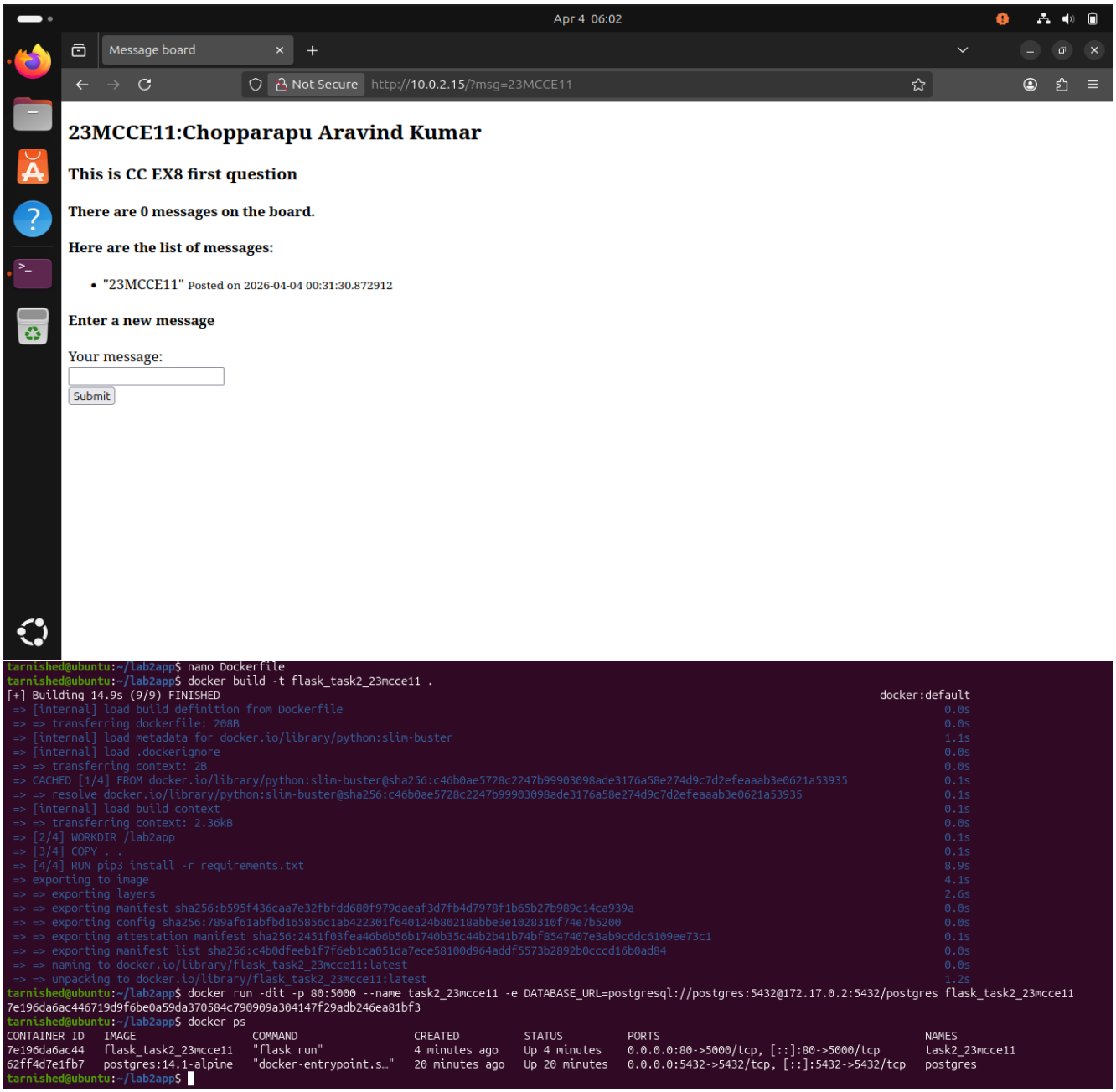
Exercise-8

Name: CH. Aravind Kumar

ID: 23MCCE11

Course: Imtech

1.

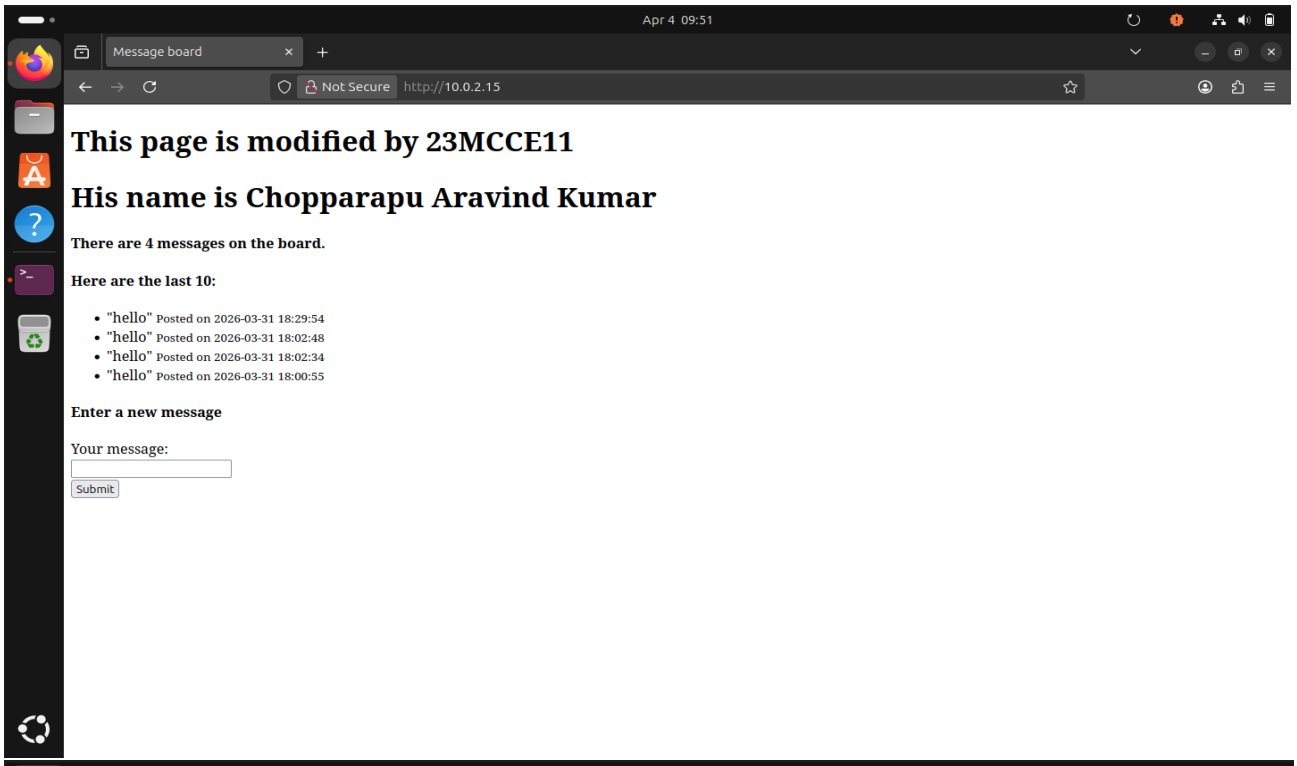


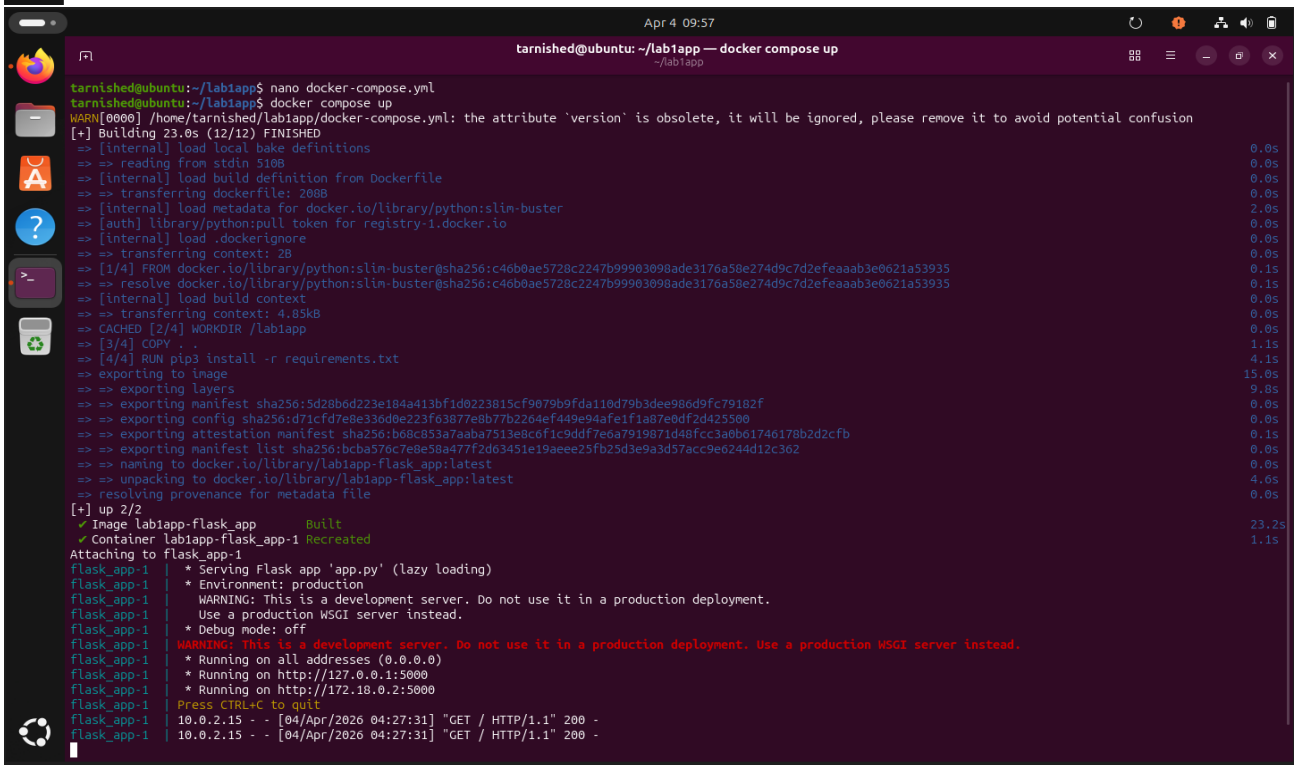

```
tarnished@ubuntu:~/lab2app$ nano Dockerfile
tarnished@ubuntu:~/lab2app$ docker build -t flask_task2_23mcce11 .
[+] Building 14.9s (9/9) FINISHED                                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 208B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:slim-buster                                                         1.1s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 2B                                                                                               0.0s
 => CACHED [1/4] FROM docker.io/library/python:slim-buster@sha256:c46b0ae5728c2247b99903098ade3176a58e274d9c7d2efeaaab3e0621a53935   0.1s
 => => resolve docker.io/library/python:slim-buster@sha256:c46b0ae5728c2247b99903098ade3176a58e274d9c7d2efeaaab3e0621a53935   0.1s
 => [internal] load build context                                                                                             0.1s
 => => transferring context: 2.36kB                                                                                           0.0s
 => [2/4] WORKDIR /lab2app                                                                                                    0.1s
 => [3/4] COPY . .                                                                                                            0.1s
 => [4/4] RUN pip3 install -r requirements.txt                                                                                8.9s
 => exporting to image                                                                                                        4.1s
 => => exporting layers                                                                                                       2.6s
 => => exporting manifest sha256:b595f436caa7e32fbfdd680f979daeaf3d7fb4d7978f1b65b27b989c14ca939a                             0.0s
 => => exporting config sha256:789af61abfbd165856c1ab422301f640124b80218abbe3e1028310f74e7b5200                               0.0s
 => => exporting attestation manifest sha256:2451f03fea46b6b56b1740b35c44b2b41b74bf8547407e3ab9c6dc6109ee73c1                 0.1s
 => => exporting manifest list sha256:c4b0dfeeb1f7f6eb1ca051da7ece58100d964addf5573b2892b0cccd16b0ad84                        0.0s
 => => naming to docker.io/library/flask_task2_23mcce11:latest                                                                0.0s
 => => unpacking to docker.io/library/flask_task2_23mcce11:latest                                                             1.2s
tarnished@ubuntu:~/lab2app$ docker run -dit -p 80:5000 --name task2_23mcce11 -e DATABASE_URL=postgresql://postgres:5432@172.17.0.2:5432/postgres flask_task2_23mcce11
7e196da6ac446719d9f6be0a59da370584c790909a304147f29adb246ea81bf3
tarnished@ubuntu:~/lab2app$ docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS                                       NAMES
7e196da6ac44   flask_task2_23mcce11  "flask run"              4 minutes ago    Up 4 minutes    0.0.0.0:80->5000/tcp, [::]:80->5000/tcp     task2_23mcce11
62ff4d7e1fb7   postgres:14.1-alpine  "docker-entrypoint.s…"   20 minutes ago   Up 20 minutes   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp  postgres
tarnished@ubuntu:~/lab2app$
```

Above image showing web page at VM_IP:80 and below one shows commands and list of running containers. I also tried removing the Postgres container and re-created it again to test volumes are persistent.

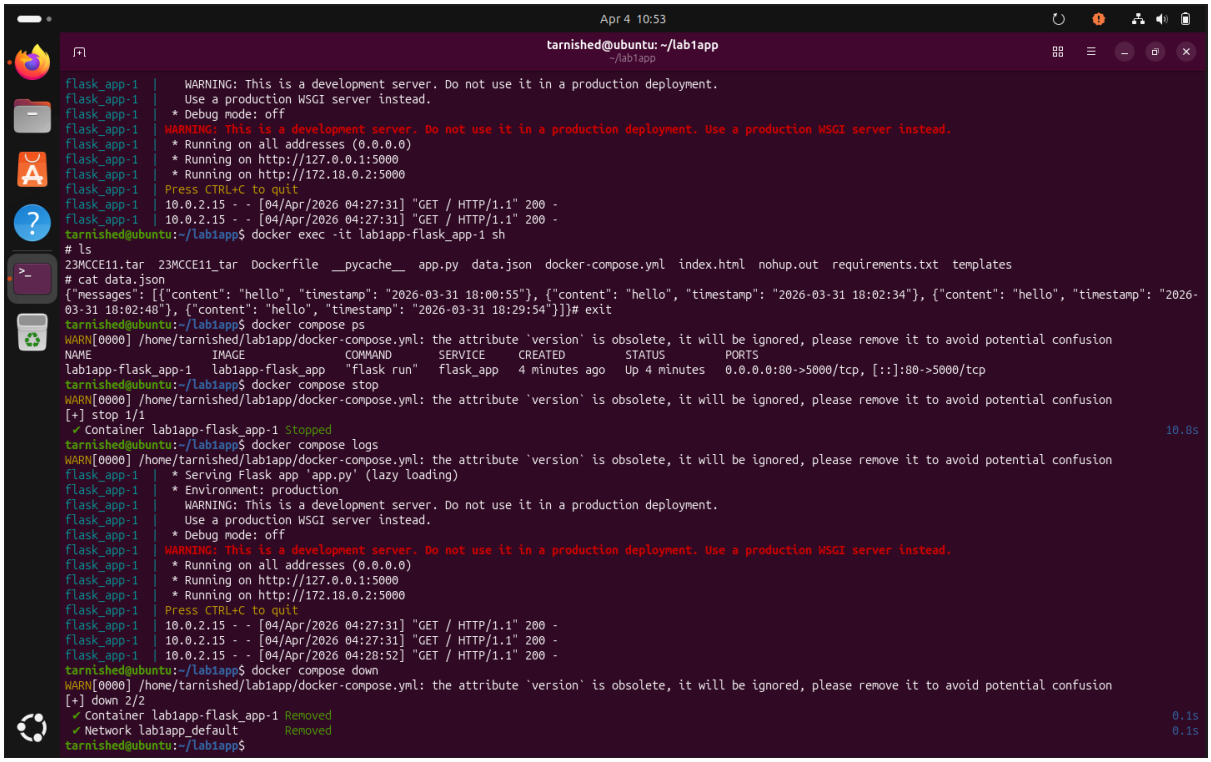
2.

2.1)

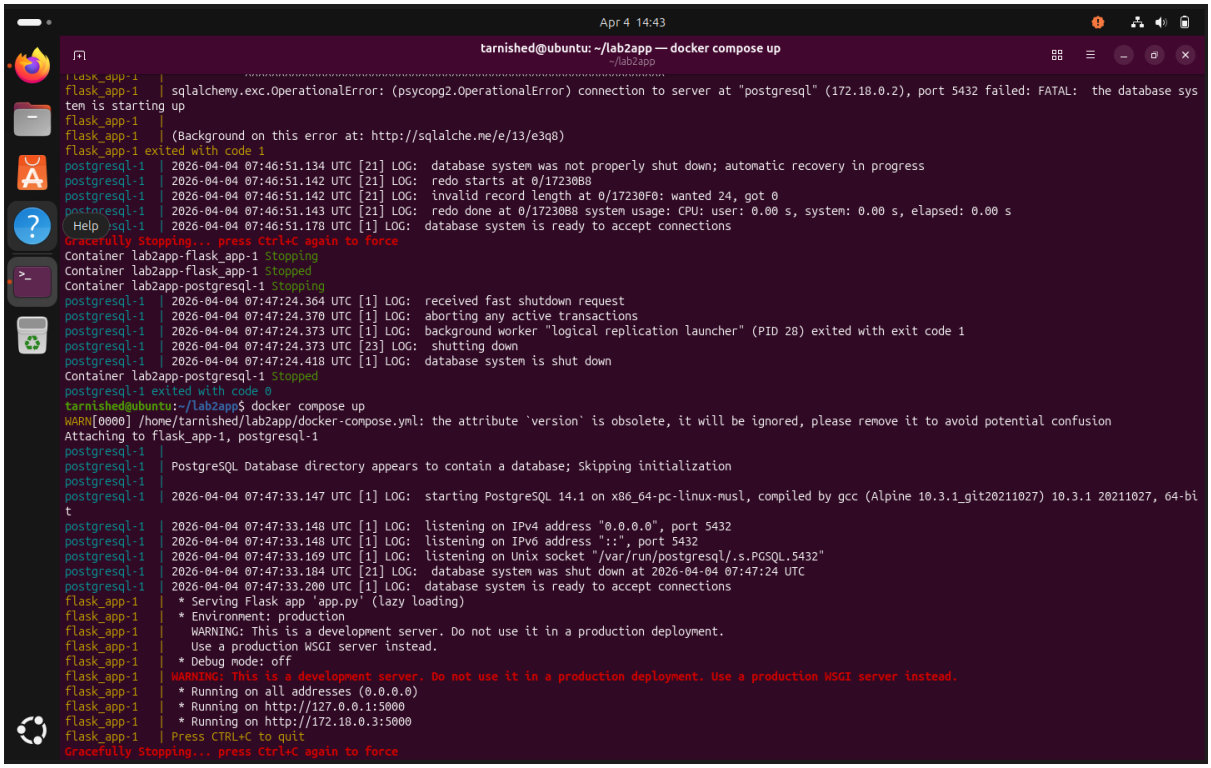




First image showing the web page and in 2nd and 3rd images show the commands I used and there are some changes in new version of docker, there is no need for '–' anymore just docker compose and once we run "docker compose up" at last we can choose to terminate it with ctrl+c or by clicking d it directly gets detached. The other 2 commands I used are "docker compose logs" to monitor the output and "docker compose down" to stop and remove the running services.

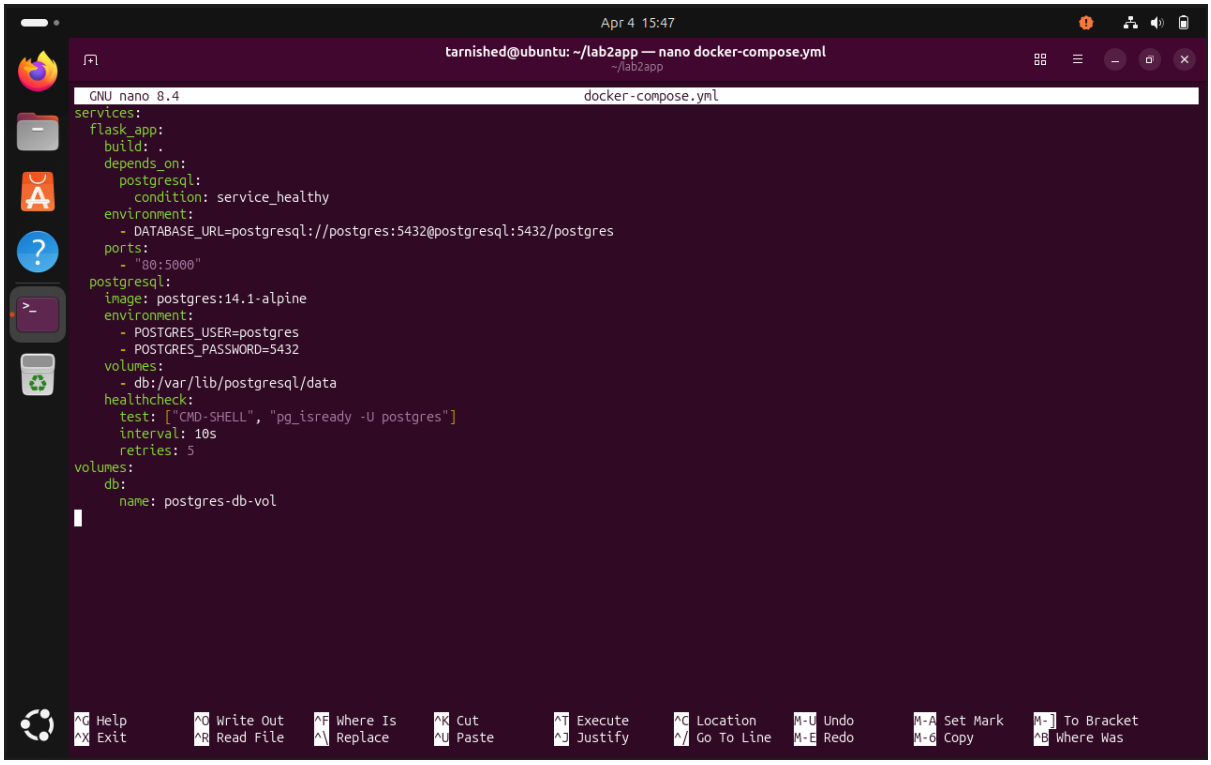


2.2

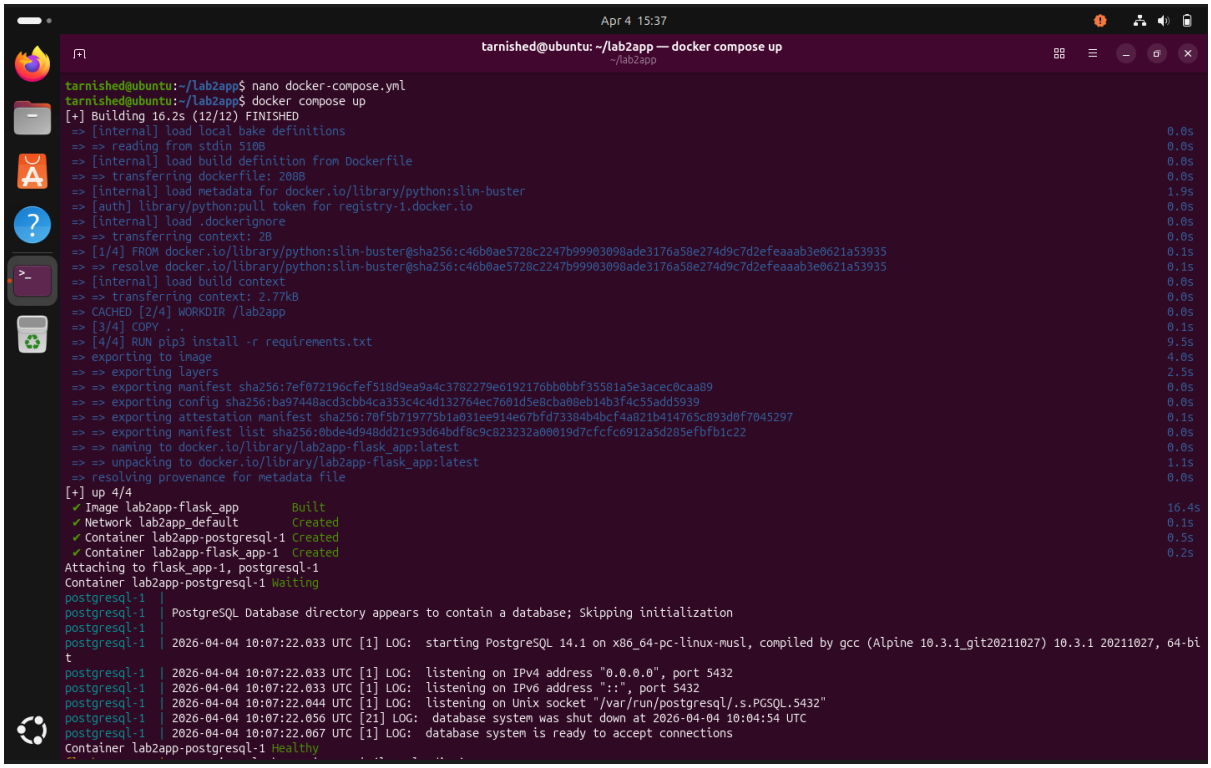


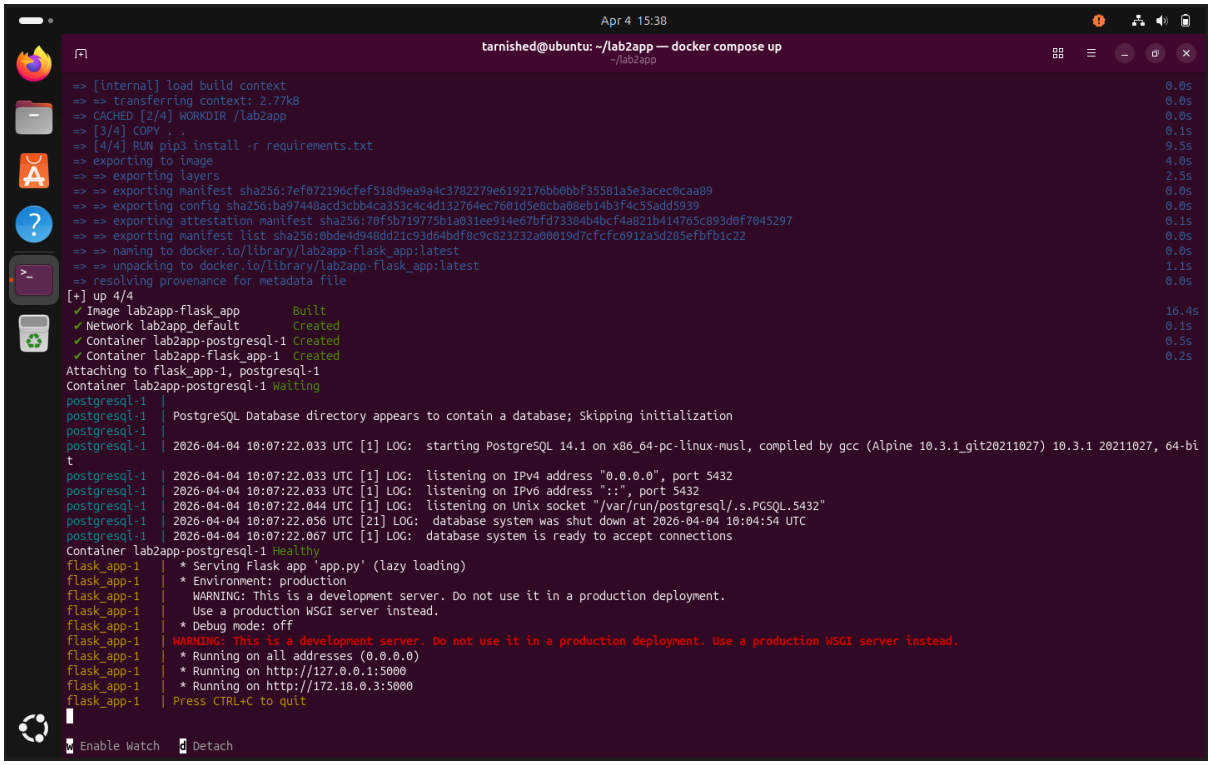
Here the flask container is not waiting for Postgres to establish connection with the data base because the data base is not ready yet. Due to this it is exiting. but if we run docker compose up command again it is running perfectly because the data base is ready. The “depends on” controls only the startup order not service readiness

(https://docs.docker.com/compose/how-tos/startup-order/). So, I am adding health checks in yaml file to ensure the db is ready before flask container starts.

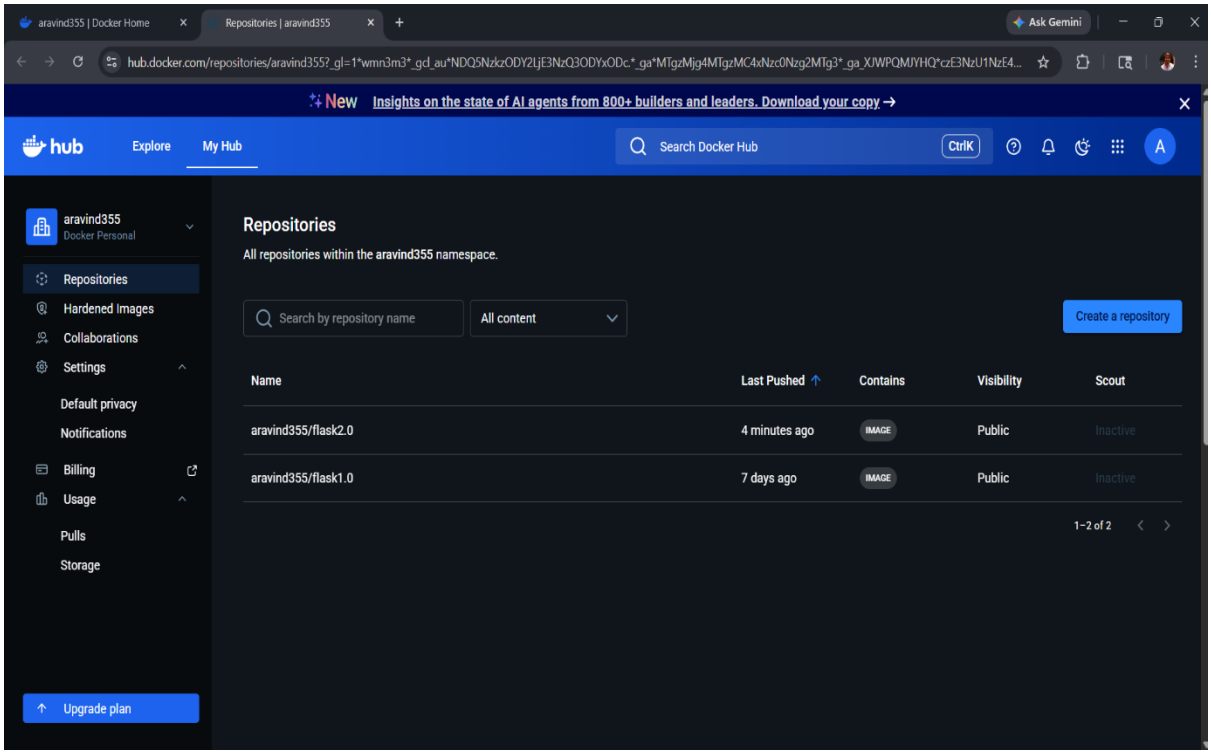


This is my final YAML file. Just added 5 extra lines. In depends_on added “condition” and in PostgreSQL added “healthcheck”.

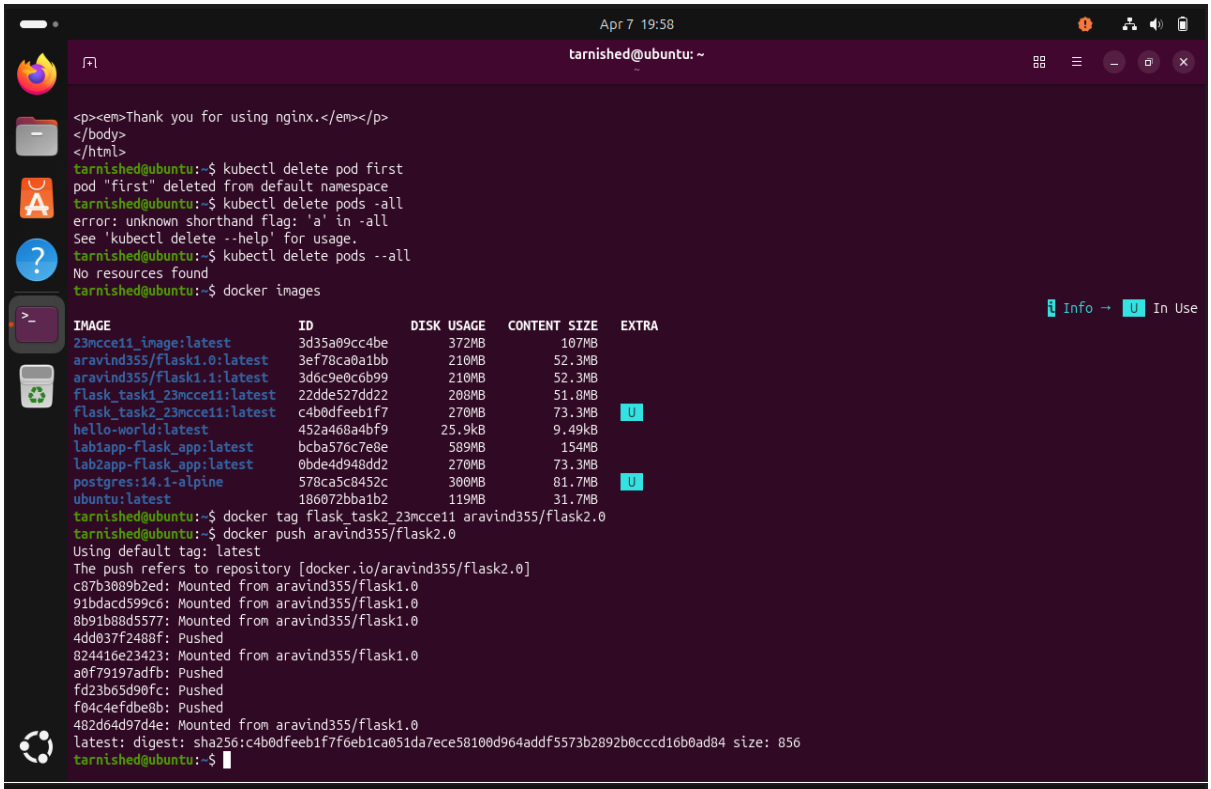


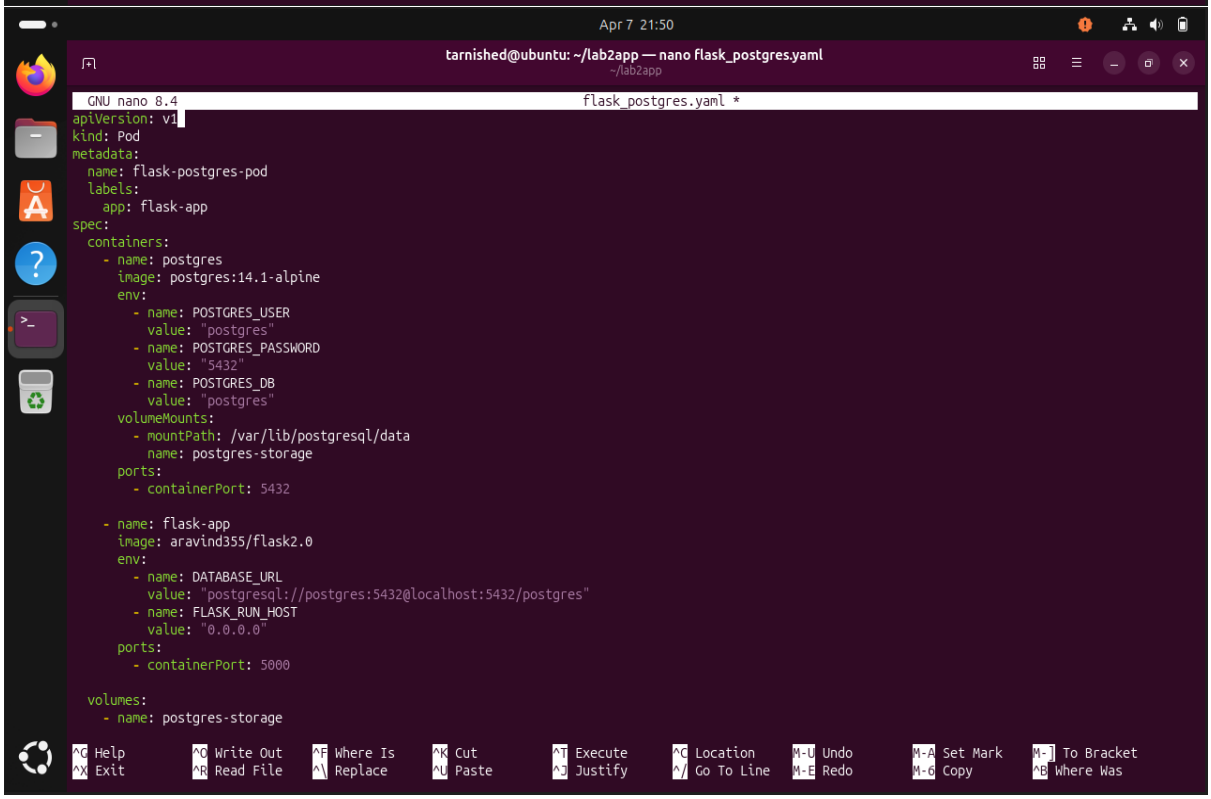


6.

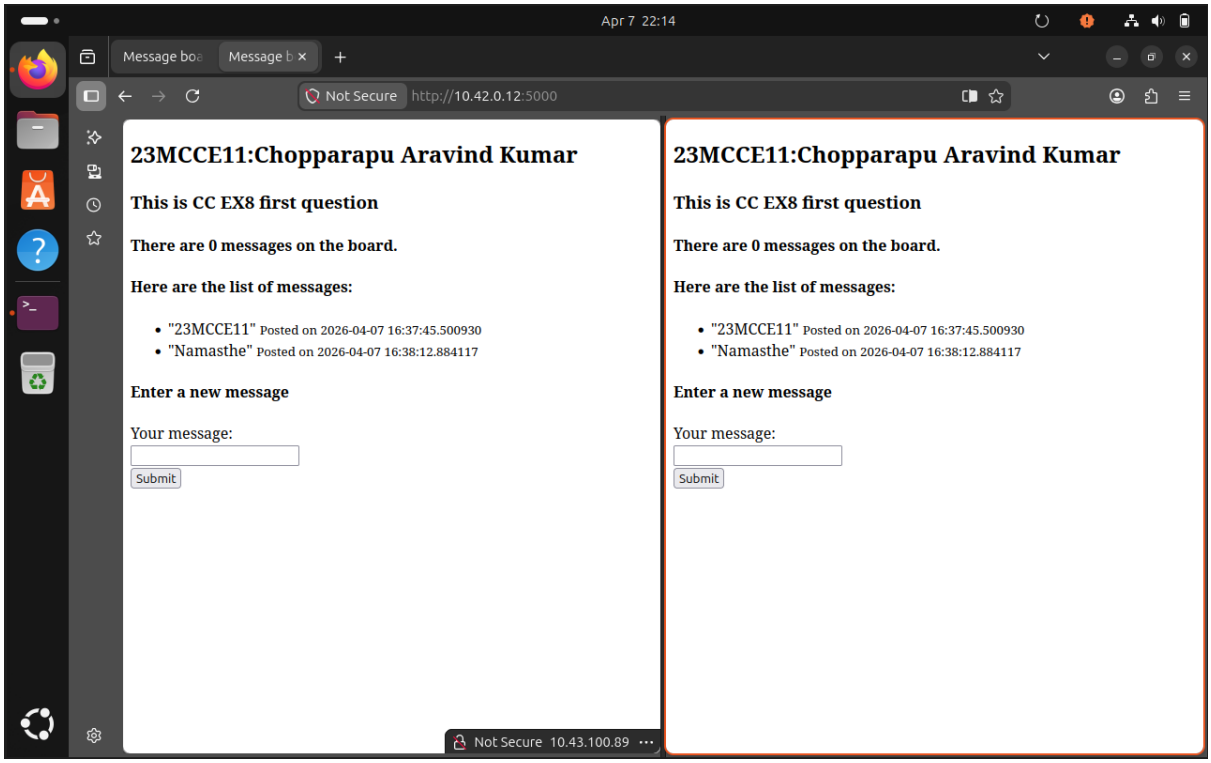


Below image showing command results.

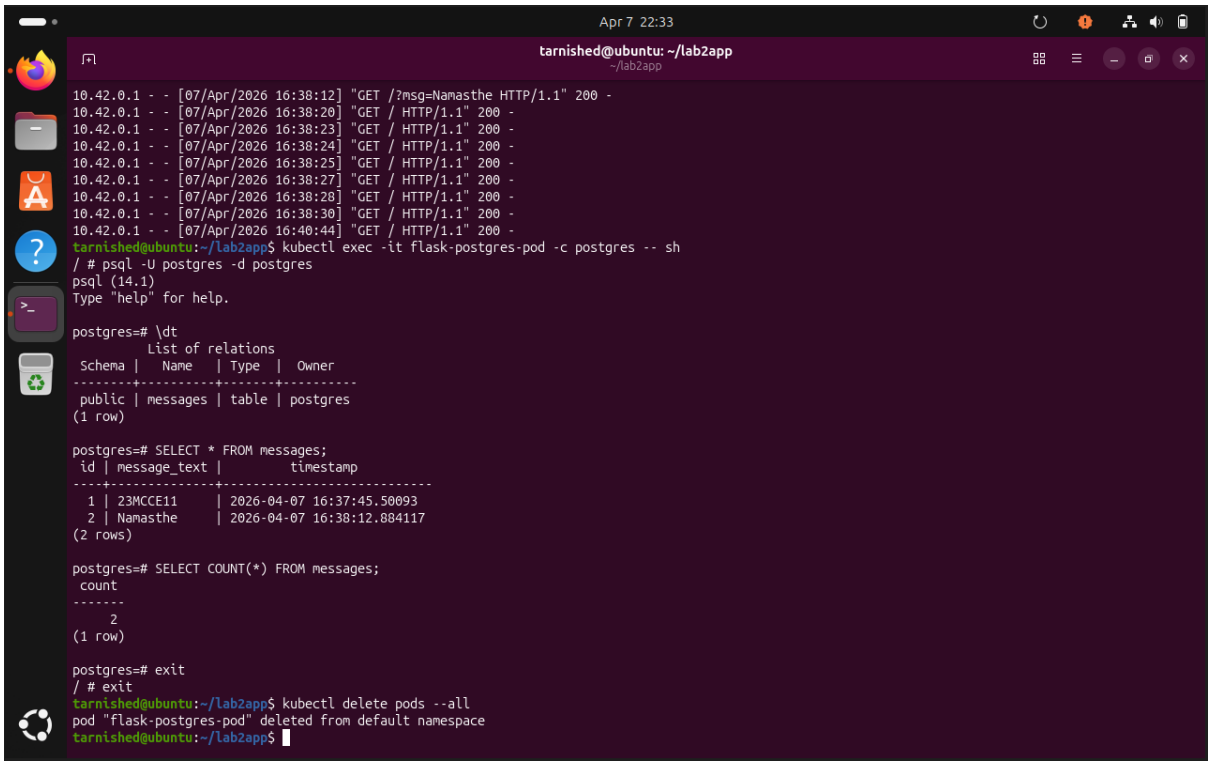




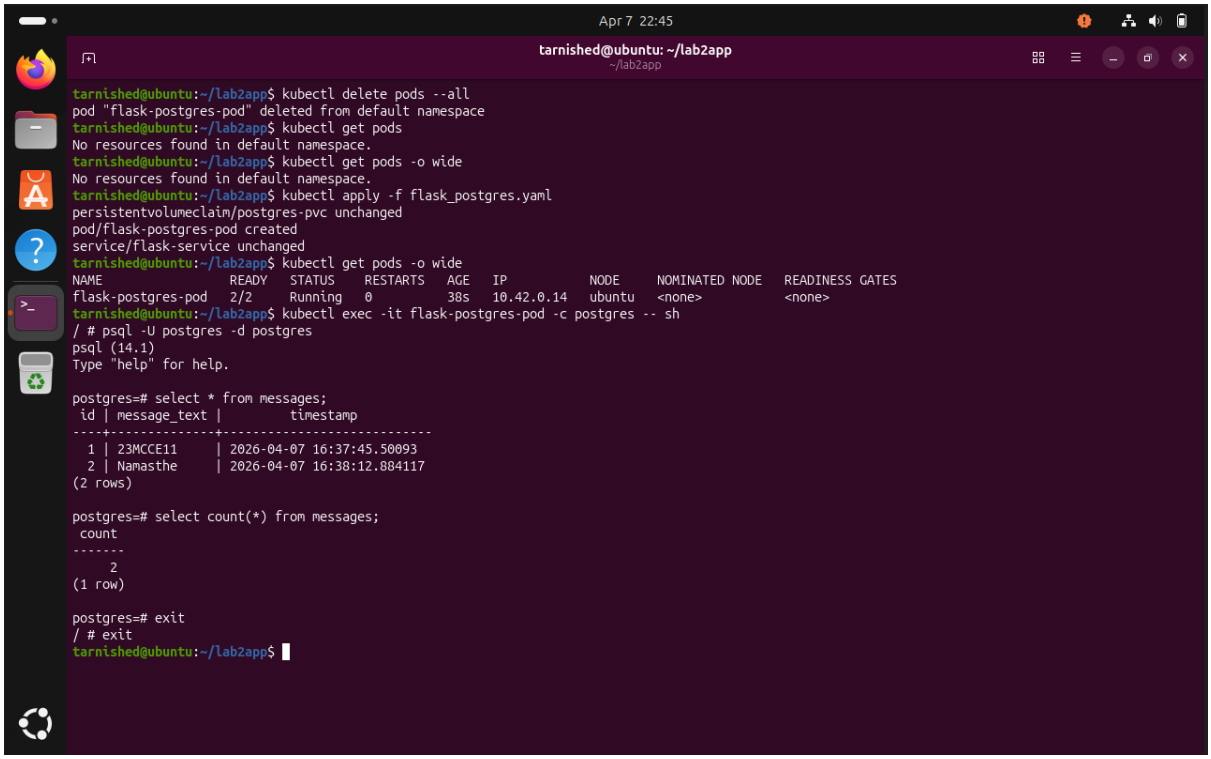
Proof that I filled the env and image.



This image showing the both tabs or IP's we can access one is node and another is ENI.



Below we can see the messages beside the pod it says created but for the other two its showing unchanged and below it we can see the values are there proving it is persistent.


Apr 7 22:45
tarnished@ubuntu: ~/lab2app
~/lab2app
tarnished@ubuntu:~/lab2app$ kubectl delete pods --all
pod "flask-postgres-pod" deleted from default namespace
tarnished@ubuntu:~/lab2app$ kubectl get pods
No resources found in default namespace.
tarnished@ubuntu:~/lab2app$ kubectl get pods -o wide
No resources found in default namespace.
tarnished@ubuntu:~/lab2app$ kubectl apply -f flask_postgres.yaml
persistentvolumeclaim/postgres-pvc unchanged
pod/flask-postgres-pod created
service/flask-service unchanged
tarnished@ubuntu:~/lab2app$ kubectl get pods -o wide
NAME READY STATUS RESTARTS AGE IP NODE NOMINATED NODE READINESS GATES
flask-postgres-pod 2/2 Running 0 38s 10.42.0.14 ubuntu <none> <none>
tarnished@ubuntu:~/lab2app$ kubectl exec -it flask-postgres-pod -c postgres -- sh
/ # psql -U postgres -d postgres
psql (14.1)
Type "help" for help.
postgres=# select * from messages;
id | message_text | timestamp
1 | 23MCCE11 | 2026-04-07 16:37:45.50093
2 | Namasthe | 2026-04-07 16:38:12.884117
(2 rows)
postgres=# select count(*) from messages;
count
2
(1 row)
postgres=# exit
/ # exit
tarnished@ubuntu:~/lab2app$